Understanding Cross-Domain Generalization in Semantic Segmentation: A Study Using Aerial Rooftop Imagery



GROUP 10

Milestone 3: Model Architecture

Course: DSAI Labs

Team: Team 10

Due: 5 November 2026

Term: September 2026

Change Log

This log is maintained for every submitted document, as required in the Milestone 1 review.

#	Item	What was done	Where	Status
1	Milestone 3 report prepared	Architecture selection, justification and end-to-end pipeline documented, following the three Milestone 3 requirements.	Sections 3–6	Incorporated
2	Dataset decisions carried forward	WHU, Inria and Massachusetts are the core datasets (23,372 prepared 512×512 tiles, Milestone 2). AIRS remains a deferred, test-only subset and is not used in this milestone.	Sections 2, 5	Incorporated
3	Protocol decisions	One model per training dataset, identical hyper-parameters, model selection only on the *source* validation split, never on target-domain data.	Sections 5, 6	Incorporated
4	Scope split with Milestone 4	Training of the initial models, the hyper-parameter, optimiser and regularisation experiments, and all measured results were moved to Milestone 4. This document keeps architecture selection, justification and pipeline set-up.	Sections 5, 6	Incorporated
5	Relative transfer drop added	A second 3×3 matrix (relative transfer drop, RTD) is defined next to the train-versus-test IoU matrix. Values are filled in Milestone 4.	Sections 5.2, 5.4	Incorporated

Contents

1 Introduction **3**

2 Problem Formulation and Requirements **3**

2.1 Task . . . 3

2.2 What the data imposes . . . 3

2.3 Requirements derived from the project . . . 3

3 Architecture Selection **4**

3.1 Candidates considered . . . 4

3.2 Comparison . . . 4

3.3 Justification of the choice . . . 5

4 The Selected Architecture: U-Net **6**

4.1 Structure . . . 6

4.2 Building blocks and parameter count . . . 7

4.3 Design choices and why . . . 7

4.4 Controlled variant . . . 8

5 End-to-End Pipeline **8**

5.1 Overview . . . 8

5.2 Stages . . . 8
5.3 Loss function and handling of class imbalance . . . 9
5.4 Metrics . . . 10
5.5 Cross-domain validity rules . . . 10
5.6 Code base and reproducibility . . . 11
5.7 Factor Sensitivity Experiments (Planned) . . . 11

6 Pipeline Verification and Hand-over **12**
6.1 Acceptance checks . . . 12
6.2 What moves to Milestone 4 . . . 12
6.3 Computational Requirements . . . 12

7 Risks and Limitations **12**

8 Summary and Next Steps **13**

A How to Run the Code **14**

1 Introduction

The project asks to what extent differences in image resolution, appearance and annotation boundaries help explain the loss of accuracy when a rooftop segmentation model trained on one dataset is tested on another (Milestone 1). Milestone 2 delivered the data: 23,372 training-ready 512×512 PNG tile pairs from WHU (aerial), Inria and Massachusetts, with one mask encoding, one folder structure and one metadata index.

The objective of Milestone 3 is the model side. It has three parts, which map to the sections of this document as follows.

Milestone 3 requirement	Where addressed
Select or design appropriate model architecture(s)	Section 3 (candidates and selection), Section 4 (the selected U-Net in detail)
Justify the choice of architecture	Section 3.3 (criteria-based justification), Section 4.3 (design choices)
Set up the complete end-to-end pipeline	Section 5 (pipeline and evaluation protocol), Section 6 (verification and hand-over), Appendix A (how to run)

2 Problem Formulation and Requirements

2.1 Task

Binary semantic segmentation: given an RGB aerial tile $x \in [0,1]^{3\times512\times512}$, predict a mask $\hat{y} \in \{0,1\}^{512\times512}$ in which 1 is building/rooftop and 0 is background.

2.2 What the data imposes

Table 1 summarises the properties of the prepared data (Milestone 2) that shape the architecture choice.

Table 1: Prepared-data properties that drive the design (from Milestone 2).

	WHU (aerial)	Inria	Massachusetts
Ground resolution	0.3 m	0.3 m	1 m
Ground area of a 512×512 tile	153.6×153.6 m	153.6×153.6 m	512×512 m
Train / val / test tiles	4,736 / 1,036 / 2,416	10,530 / 2,025 / 2,025	548 / 16 / 40
Building share of pixels (train)	18.66%	16.38%	13.24%
Empty tiles (train)	8.85%	14.99%	3.83%
Label source	hand-edited footprints	official footprints	OpenStreetMap footprints

2.3 Requirements derived from the project

1. **R1 – Constant, interpretable baseline.** The research question is about the *data* (resolution, appearance, labels). The architecture must therefore be held fixed across all experiments, and it should be simple enough that a performance drop can be attributed to the data rather than to an exotic model component.

2. **R2 – Pixel-accurate boundaries.** Annotation-boundary differences between datasets are one of the studied factors, so the model must be able to reproduce fine building outlines, not only coarse blobs.

3. **R3 – Works on small and large training sets.** Training sizes range from 548 tiles (Massachusetts) to 10,530 tiles (Inria); the same architecture must train sensibly on both.

4. **R4 – Handles class imbalance.** Buildings cover only 11–19% of pixels, and up to 29% of validation tiles contain no building.

5. **R5 – Comparable with the literature.** Results should be interpretable against published building-extraction work on these datasets.

6. **R6 – Practical.** It must train on a single GPU of a team member and be reproducible from a small code base.

7. **R7 – No hidden confounds.** Components that would themselves change cross-domain behaviour (for example pre-training on other imagery) must not be part of the baseline, because appearance shift is a studied factor.

3 Architecture Selection

3.1 Candidates considered

- **FCN** (Long et al., 2015) [2]: the first fully convolutional segmentation network; coarse output stride recovered by simple up-sampling and summed skip connections.
- **U-Net** (Ronneberger et al., 2015) [1]: a symmetric encoder–decoder with a skip connection (concatenation) at every resolution level.
- **U-Net with a pre-trained ResNet-34 encoder** [6, 7]: the same decoder idea, with an ImageNet-pre-trained residual encoder.
- **DeepLabv3+** (Chen et al., 2018) [3]: atrous spatial pyramid pooling for multi-scale context, with a light decoder to recover boundaries; the encoder normally runs at output stride 16.
- **SegFormer** (Xie et al., 2021) [4]: a hierarchical transformer encoder with a light all-MLP decoder; transformer encoders are normally used with pre-training [5].

3.2 Comparison

Table 2 compares the candidates against requirements R1–R7. The ratings are qualitative and are based on the cited literature and on the design of each network; they are *not* measurements made by this team.

Table 2: Qualitative comparison of candidate architectures against the project requirements.

Candidate	Boundary detail (R2)	Small-data behaviour (R3)	Confounds / complexity (R1, R7)	Verdict
FCN	Weaker: coarse stride, limited fine skip detail	Usually needs a pre-trained backbone	Pre-trained backbone is typical	Not chosen
U-Net (scratch)	**Strong: full-resolution skip at every level**	**Designed for few training images; trains from scratch**	**No pre-training, one simple design**	**Baseline**
U-Net + ResNet-34 (pre-trained)	Strong	Strong, helped by pre-training	Adds ImageNet pre-training, itself a possible factor in appearance robustness	Variant (Milestone 4)
DeepLabv3+	Good, but starts from stride-16 features	Typically pre-trained	More components (ASPP, atrous convolutions) to reason about	Future work
SegFormer	Moderate; depends on patch size and decoder	Typically needs pre-training and more data	Heaviest; many design choices	Future work

Summarising the merits and trade-offs of each candidate,

- FCN is the weakest candidate, as its coarse stride and upsampling results in blurry edges, which may confound the boundary analysis.
- U-Net is a good choice for a baseline due to the robustness provided by its skip connections, as well as its lightweight architecture. However, the fixed receptive field makes it scale-sensitive.
- U-Net with a pretrained ResNet-34 adds ImageNet pre-training as a potentially confounding factor. However, it is still worth considering as a variant to combat underfitting when training on Massachusetts, due to a small dataset.
- DeepLabv3+ uses Atrous Spatial Pyramid Pooling (ASPP) to view the image through multiple receptive fields at the same time while maintaining a high resolution. However, it starts from stride-16 features, and due to its complex architecutre, its configuration space is very broad.
- SegFormer uses a Vision Transformer architecture, which makes it powerful but inherently data hungry. It also has the broadest set of design choices, making it a weaker candidate for a reproducible cross-dataset pipeline.

3.3 Justification of the choice

The plain U-Net trained from scratch is selected as the **primary architecture**, for the following reasons.

(1) It keeps the experiment about the data (R1, R7). The study varies the training and test dataset and, later, resolution, appearance and label definition. If the model were a large pre-trained network, an observed drop could partly come from how pre-training interacts with the appearance of each dataset. A from-scratch U-Net contains no knowledge from outside the training dataset, so what it learns, and what it fails to transfer, comes from that dataset alone.

(2) Skip connections serve the boundary question (R2). Each decoder stage receives the encoder feature map of the same spatial size by concatenation. Fine spatial detail from the early layers therefore reaches the output directly, which is what is needed to follow roof edges. Since differences in how roof boundaries are annotated (hand-edited, official, OpenStreetMap) are a studied factor, a model that can represent boundaries sharply is a prerequisite: otherwise label-boundary effects would be hidden by the model's own blur.

(3) It suits very different training sizes (R3). The original U-Net was designed for biomedical segmentation with very few annotated images, relying on augmentation and on the dense skip structure [1]. Massachusetts, with only 548 training tiles, is closest to that regime; WHU and Inria have many more tiles and are not limited by it.

(4) It is a standard in building extraction (R5). U-Net and its variants are widely used for building extraction from aerial imagery, and the benchmarks used here originate from work on convolutional encoder–decoder style segmentation [11, 12]. A U-Net baseline therefore places our numbers in a familiar frame of reference.

(5) It is practical (R6). The network has about 31.0 M parameters (counted from the layer definitions in Section 4) and needs no external weights. Mixed precision, a smaller batch size and optional random crops are available if GPU memory is limited.

(6) It is a better baseline than the alternatives:

- While FCN is simpler, U-Net's skip connections make it better at preserving boundary detail, making it better for comparing dataset performance and measuring generalisability.
- U-Net with ResNet-34 may add its ImageNet pretraining as a confounding factor; although it is still an alternative worth considering for a potentially stronger baseline.
- U-Net's simpler architecture means less hyperparameters to tune than DeepLabv3+, making it a better baseline model for a reproducible pipeline.
- SegFormer requires large volumes of data due to its Transformer architecture, which may not always be feasible, with Indian rooftop imagery datasets. U-Net in comparison provides reasonable benefits without needing too much data.

In summary, pre-trained and more elaborate networks (ResNet-34 U-Net, DeepLabv3+, SegFormer) would likely reach higher absolute scores, but they add components whose own effect on cross-domain behaviour is unknown, which conflicts with R1 and R7. The goal of the project being a clean measurement of degradation - rather than higher accuracy - U-Net from scratch proves to be the better baseline. The ResNet-34 U-Net is nevertheless included as a *controlled variant* that is evaluated in Milestone 4, so that the effect of pre-training on the degradation can itself be measured.

4 The Selected Architecture: U-Net

4.1 Structure

Figure 1 shows the network. The encoder halves the spatial size four times by 2×2 max-pooling while doubling the number of channels (64, 128, 256, 512, 1024). The decoder reverses this with 2×2 transposed convolutions, and after each up-sampling the encoder feature map of the same size is concatenated before a double convolution. A final 1×1 convolution produces one logit per pixel; a sigmoid turns it into a building probability.

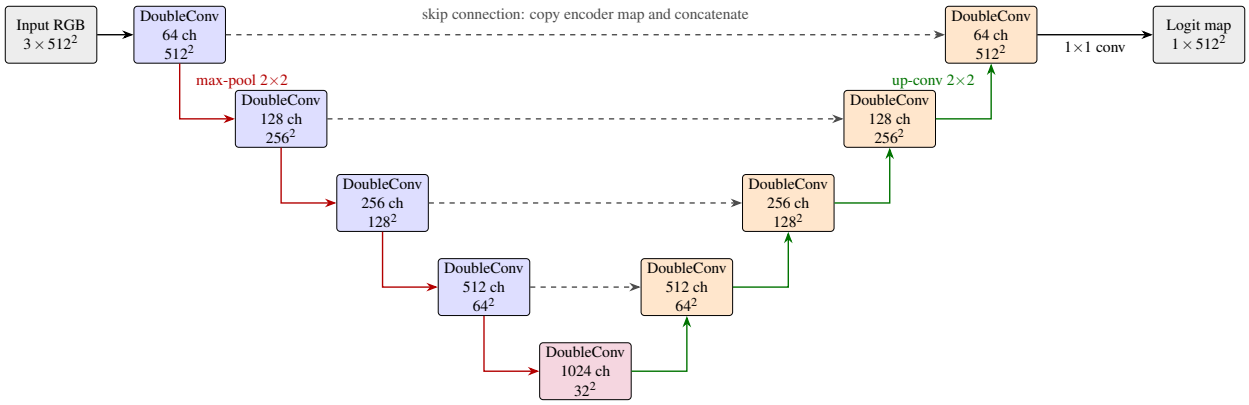


Figure 1: U-Net used as the baseline. Each "DoubleConv" block is Conv3×3–BatchNorm–ReLU applied twice. Channel counts are those of the block output; the decoder blocks receive twice that many channels after concatenation (for example 1024 channels into the first decoder block, 512 coming from the up-convolution and 512 from the skip). Spatial sizes are for a 512×512 input.

4.2 Building blocks and parameter count

- **DoubleConv:** Conv3×3 (padding 1, no bias) → BatchNorm [9] → ReLU, twice.
- **Down:** 2×2 max-pool followed by DoubleConv.
- **Up:** 2×2 transposed convolution (stride 2) that halves the channels, concatenation with the skip map, DoubleConv.
- **Head:** 1×1 convolution to one channel (logit). The sigmoid is folded into the loss and the threshold during evaluation.

Counting weights from these definitions with 64 base channels gives **31,037,633 parameters (≈31.0 M)**, of which the 1024-channel bottleneck block alone has about 14.2 M.

4.3 Design choices and why

Table 3: Design choices of the baseline U-Net.

Choice	Setting	Reason
Padding	"same" (padding 1)	Output has the same size as the input, so the prediction aligns pixel-for-pixel with the mask and no cropping is needed, which keeps the metrics simple.
Normalisation	BatchNorm	Stable training from scratch at the chosen learning rate. Note that BatchNorm stores source-dataset statistics; this is part of what is tested when the model meets another domain (Section 7).
Up-sampling	Transposed convolution	Learnable up-sampling that follows the original design.
Output	1 logit per pixel	Binary task; avoids a redundant second class channel.
Dropout	None	Data augmentation and early stopping are used for regularisation instead.
Depth	4 down-sampling stages	The bottleneck sees a 32×32 grid, i.e. the whole tile at coarse scale.

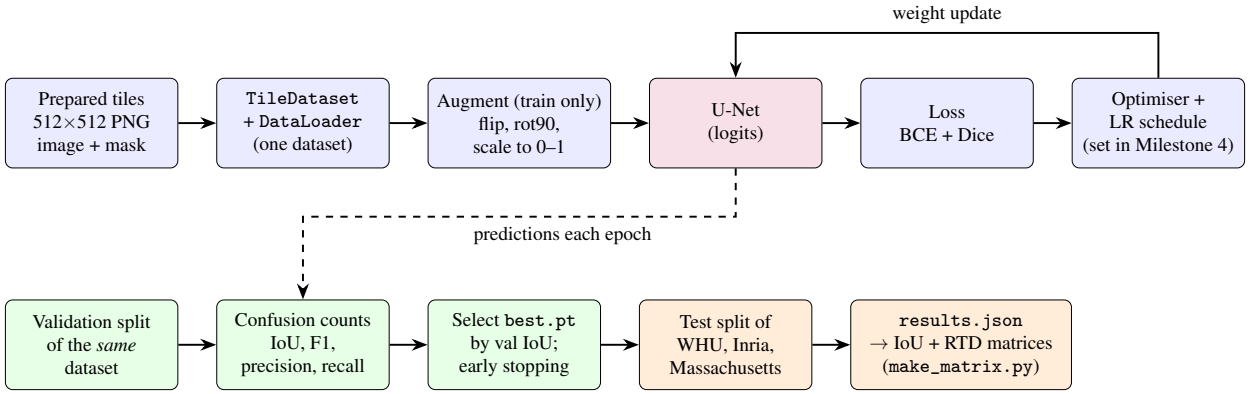


Figure 2: End-to-end pipeline. Model selection uses only the validation split of the training dataset; the test splits of all three datasets are used once, after training has finished.

Receptive field and resolution. Following the standard recursion $r \leftarrow r + (k-1)\,j$ (kernel k, jump j), the theoretical receptive field of the bottleneck is about 140 input pixels. At 0.3 m per pixel (WHU, Inria) that is roughly 42 m of ground, while at 1 m per pixel (Massachusetts) the same network looks at about 140 m. The same architecture therefore sees different amounts of real-world context in the two resolution groups. This is not a flaw of the choice but a useful reminder when interpreting the resolution factor of the study.

4.4 Controlled variant

A second configuration, selected with `--arch smp_unet --pretrained`, uses a U-Net whose encoder is ResNet-34 [6] with ImageNet weights, implemented through `segmentation_models_pytorch` [8] (the approach follows [7]). ImageNet mean and standard deviation are applied *inside* the model so that the stored data and the data pipeline remain the plain 0–1 scaling of Milestone 2. This variant is not the baseline; it is evaluated as a controlled comparison in Milestone 4.

5 End-to-End Pipeline

5.1 Overview

Figure 2 shows the complete pipeline. The upper row is training on one dataset; the lower row is validation, model selection and the cross-dataset test that produces one row of the train-versus-test matrix.

End-to-End Workflow. The pipeline is operated by team members through command-line scripts, with no separate user interface. Each run takes the prepared image-mask pairs from one dataset as input and follows a sequence of steps: data loading (`dataset.py`), training-time augmentation, U-Net prediction (`model.py`), validation-based checkpoint selection, testing on all three datasets, and result aggregation into the IoU and RTD matrices (`make_matrix.py`).

The outputs, including the configuration, training log, best and last checkpoints, and results file, are saved under `runs/` on the team server. Before training, `check_data.py` verifies the dataset, while a dry run checks that the full pipeline works (Section 6.1). If GPU memory is insufficient, a smaller batch size or random crop can be used. Interrupted training can also be resumed from the last saved checkpoint.

5.2 Stages

1. **Data loading (`dataset.py`).** Tiles are found under `prepared_dataset/` with one folder per dataset (WHU, Inria, Mass), then train, validation and test, each holding `images/` and `masks/` (Milestone 2

layout). Each image is paired with the same-named mask. Masks are binarised (`>0`) as a safeguard, images are read as 3-channel RGB.

2. **Pre-processing and augmentation.** Pixel values are scaled to 0–1; nothing else is done to appearance (no colour correction, equalisation or sharpening), exactly as fixed in Milestone 2, because appearance is a studied factor. At training time only, random horizontal flips, vertical flips and 90° rotations are applied identically to image and mask. An optional random square crop (`--crop`) reduces memory use. Validation and test tiles are never augmented.

3. **Model (`model.py`).** The U-Net of Section 4, or the pre-trained variant.

4. **Loss.** The sum of binary cross-entropy and soft Dice loss (Section 5.3).

5. **Optimisation (`train.py`).** Optimiser, learning-rate schedule, regularisation and early stopping are selected through command-line options; their values are chosen and justified in Milestone 4. The best checkpoint by validation IoU is stored, and the last state is saved each epoch so that an interrupted run can be resumed.

6. **Evaluation (`evaluate.py`, `metrics.py`).** The best checkpoint is applied to the test split of every dataset with the same code and the same metrics.

7. **Aggregation (`make_matrix.py`).** All `results.json` files are merged into the train-versus-test IoU matrix, the relative transfer drop matrix (Section 5.4) and a CSV file.

5.3 Loss function and handling of class imbalance

Why BCE + Dice? BCE (i.e. Binary Cross-Entropy) on its own would judge every single pixel, and is stable to train. However, in rooftop imagery, the class is inherently heavily imbalanced due to background pixels dominating most of the image. When using only BCE on a heavily imbalanced dataset, the model could achieve very high performance simply by predicting every single pixel as the background.
Dice loss is designed specifically to deal with imbalanced datasets, as it measures and maximises the region overlap between a predicted segmentation mask and the ground-truth mask, thus treating the foreground object - i.e. the rooftops - as an entire region, rather than a collection of independent pixels. As per the formula presented below, if the model were to predict background class for all pixels, then the Dice coefficient would be 0, and thus the loss value would be 1.0, i.e. the maximum possible loss.
However, Dice lacks the stability of BCE. Thus, by combining BCE and Dice, the training would remain stable while also accounting for the class imbalance.

Let $p_i = \sigma(z_i)$ be the predicted building probability of pixel i and $y_i \in \{0,1\}$ the label. For a batch of tiles the loss is

$$\mathscr{L} = \mathscr{L}_{\mathrm{BCE}} + \mathscr{L}_{\mathrm{Dice}}, \tag{1}$$

$$\mathscr{L}_{\mathrm{BCE}} = -\frac{1}{N}\sum_{i}\left[y_i \log p_i + (1-y_i)\log(1-p_i)\right], \tag{2}$$

$$\mathscr{L}_{\mathrm{Dice}} = 1 - \frac{1}{B}\sum_{b=1}^{B}\frac{2\sum_{i\in b} p_i y_i + \varepsilon}{\sum_{i\in b} p_i + \sum_{i\in b} y_i + \varepsilon}, \qquad \varepsilon = 1, \tag{3}$$

where N is the number of pixels in the batch and B the number of tiles. Because buildings are only 11–19% of pixels, cross-entropy alone is dominated by the background and can reward predicting too little building. The Dice term measures overlap directly and is largely insensitive to the number of background pixels [10]; the cross-entropy term gives smooth gradients at the start of training. The smoothing constant ε keeps the loss defined on empty tiles, where it pushes the predicted probabilities towards zero. No class weights or tile re-sampling are used in the baseline: empty tiles are kept in training as they occur, and the imbalance is documented rather than corrected, so that every dataset is treated by the same rule. Alternative losses are compared in Milestone 4.

```
def dice_loss(logits, target, eps=1.0):
    p = torch.sigmoid(logits)
    inter = (p * target).sum((1, 2, 3))
    denom = p.sum((1, 2, 3)) + target.sum((1, 2, 3))
    return 1 - ((2 * inter + eps) / (denom + eps)).mean()

class BCEDice(nn.Module):
    def __init__(self):
        super().__init__(); self.bce = nn.BCEWithLogitsLoss()
    def forward(self, logits, target):
        return self.bce(logits, target) + dice_loss(logits, target)
```

Listing 1: Loss implementation (train.py).

5.4 Metrics

Predictions are thresholded at 0.5 (fixed, never tuned on target data). True positives, false positives and false negatives are accumulated over *all pixels of the whole evaluation set* and the metrics are computed once at the end:

$$\text{IoU} = \frac{TP}{TP+FP+FN}, \quad \text{Precision} = \frac{TP}{TP+FP}, \quad \text{Recall} = \frac{TP}{TP+FN}, \quad F_1 = \frac{2TP}{2TP+FP+FN}.$$

Accumulating counts first, rather than averaging per-tile scores, avoids undefined or misleading values on empty tiles, which are kept in the test sets (Milestone 2). Pixel accuracy is not reported because background dominates. IoU is the primary metric. Along with standard IoU, a boundary-tolerant IoU will be reported in Milestone 5. It allows small differences in building outlines without penalising them as heavily. Comparing both scores will help us understand whether errors mainly occur around building boundaries or come from missed buildings. The tolerance width will be fixed and kept the same across all datasets. This metric will be used only for evaluation, not model selection.

Relative transfer drop. Absolute IoU values are not directly comparable across training sets, because the in-domain accuracy of each model differs. Alongside the train-versus-test IoU matrix we therefore report a second 3×3 matrix, the relative transfer drop. For a model trained on dataset A and tested on dataset B,

$$\text{RTD}(A \to B) \;=\; \frac{\text{IoU}(A,A) - \text{IoU}(A,B)}{\text{IoU}(A,A)} \times 100\%, \tag{4}$$

where $\text{IoU}(A,B)$ is the test IoU of the model trained on A and evaluated on the test split of B. Each row is normalised by the in-domain (diagonal) score of its own training dataset, so the diagonal entries are 0% by definition and a larger value means a larger loss of accuracy when the model meets a new domain. A negative value would mean that the model scores higher on the other dataset than on its own. The matrix is computed from the same `results.json` files by `make_matrix.py`, so no extra evaluation is needed.

Table 4: Relative transfer drop (%) of the baseline U-Net, Eq. (4). Rows: training dataset; columns: test dataset. Off-diagonal values are filled in Milestone 4.

Train \ Test	WHU	Inria	Massachusetts
WHU	0	–	–
Inria	–	0	–
Massachusetts	–	–	0

5.5 Cross-domain validity rules

These rules make the 3×3 matrix a fair measurement of degradation:

1. Architecture, loss, optimiser, schedule, augmentation, evaluation code and seed are identical for all runs; only the training dataset changes.

2. Early stopping and checkpoint selection use *only* the validation split of the training dataset. Neither the test split of the same dataset nor any other dataset is used for selection or tuning.

3. The test splits of all datasets are evaluated once, with the checkpoint selected in rule 2.

4. Splits are those fixed in Milestone 2 (official for WHU and Massachusetts; documented same-city split for Inria), cut at source-image level before tiling, so no leakage exists between splits.

5.6 Code base and reproducibility

```
unet_rooftop/
  dataset.py      tile pairs, 0-1 scaling, flip/rot90 augmentation
  model.py        U-Net (scratch) and optional ResNet34-encoder U-Net
  metrics.py      TP/FP/FN-based IoU, precision, recall, F1
  train.py        training, model selection, final test on all datasets
  evaluate.py     evaluate any checkpoint on any split
  make_matrix.py  collect runs into the IoU and RTD matrices
  check_data.py   data and forward-pass sanity check
  run_all.sh      the three baseline runs
  requirements.txt, README.md
```

Every run stores its configuration (`config.json`), a per-epoch log (`log.csv`), the best and last checkpoints, and `results.json`. Seeds are fixed for Python, NumPy and PyTorch. Because cuDNN auto-tuning and mixed precision are enabled for speed, two runs with the same seed are expected to be very close but not necessarily bit-identical; the repetition over several seeds (Milestone 4) is what quantifies this.

5.7 Factor Sensitivity Experiments (Planned)

Three experiments are planned for Milestone 5 to study how resolution, annotation boundaries and image appearance affect model performance. The trained models will remain fixed; only the test data will be modified.

1. **Resolution:** Test images will be resampled to different resolutions to check how this affects the model's performance.

2. **Annotation Boundaries:** Test masks will be shifted, eroded and dilated to study how small changes in building outlines affect IoU.

3. **Appearance:** The brightness and colours of test images will be changed to see how sensitive the model is to visual differences.

These experiments will help us understand the model's sensitivity to each factor, as discussed in Objective 1 of Milestone 1. However, they cannot completely isolate the effect of each factor or prove that it alone causes cross-domain performance drops.

The baseline U-Net is trained from scratch on one dataset at a time. All layers are trainable, what the model learns comes only from its training dataset. The loss function is BCE + Dice, as described in Section 5.3. The optimiser, learning-rate schedule, batch size, number of epochs and early-stopping patience are taken as command-line options in train.py. Their values will be fixed before the first Milestone 4 training run and will then be held identical for all three training datasets, so that only the training dataset changes between runs (Section 5.5, Rule 1). Model selection and early stopping use only the validation split of the training dataset, based on validation IoU. The best checkpoint is saved as best.pt, and the last state is saved every epoch so that an interrupted run can be resumed. No target-domain data is used for training, tuning or selection.

6 Pipeline Verification and Hand-over

6.1 Acceptance checks

The pipeline is accepted for hand-over to training when all of the following hold.

1. `check_data.py` finds all nine dataset/split folders, the tile counts equal those of Milestone 2 (WHU 8,188, Inria 14,580, Massachusetts 604), and every mask contains only the values 0 and 1.
2. A dry run (`train.py --dry_run`) completes end to end: data loading, forward and backward pass, validation, checkpointing and the final test evaluation on all three datasets.
3. Evaluating the same checkpoint twice gives identical metrics (the evaluation path is deterministic: no augmentation, evaluation mode, fixed threshold).
4. Metrics behave as expected on trivial cases: an all-background prediction gives IoU 0 on non-empty data.

6.2 What moves to Milestone 4

Everything that involves actually training models is reported in Milestone 4: the initial models and their train-versus-test matrix, seed repeats, the equal-budget control and the pre-trained variant, the experiments on hyper-parameters, optimisation methods and regularisation, and the final configuration. This document defines the model and the pipeline those experiments run on.

6.3 Computational Requirements

All training and evaluation will be done using the team's available resources on a single GPU, without any external services or APIs. The baseline U-Net has approximately 31.0 million parameters and takes a 512×512 RGB tile as input. If GPU memory becomes an issue, mixed precision, smaller batch sizes and random cropping can be used (Appendix A).

Storage is required for the prepared dataset (23,372 image-mask pairs) and the outputs of each run, including configurations, logs, results and model checkpoints. These are stored under `prepared_dataset/` and `runs/` on the team server.

Inference requires one forward pass per tile followed by thresholding at 0.5. Its execution time has not been measured yet and will be reported in Milestone 4.

7 Risks and Limitations

- **Training-dependent limitations** (hyper-parameter choice, training-set size differences, the 16-tile Massachusetts validation set) are treated in Milestone 4.
- **Mitigation for the Massachusetts validation set.** Since the Massachusetts validation set contains only 16 tiles, its IoU can change significantly based on just one or two tiles. To reduce this risk, checkpoint selection will consider validation scores across neighbouring epochs rather than a single score. Training will also be repeated with different seeds in Milestone 4, and the variation in results will be reported. The original Milestone 2 splits will remain unchanged, and the test set will never be used for model selection. However, checkpoint selection for Massachusetts will remain less reliable than for WHU and Inria, so its results will be interpreted with caution.
- **Inria in-dataset scores are optimistic.** The split is same-city (Milestone 2), so its diagonal entry will be higher than for an unseen city.

- **BatchNorm statistics are source-specific.** At test time the model uses running statistics from its training dataset. Part of the observed degradation may therefore come from normalisation statistics rather than from learned features. This is a possible additional factor and is not separated in the baseline.
- **Fixed resolution of context.** As computed in Section 4.3, the same receptive field in pixels corresponds to different ground areas for 0.3 m and 1 m data.
- **Conclusions about causes are limited.** In line with the Milestone 1 research question, the experiments measure sensitivity to selected factors; they do not isolate each factor perfectly.
- **Reproducibility is approximate, not bit-exact,** for the reasons given in Section 5.6.

8 Summary and Next Steps

A plain U-Net trained from scratch was selected as the baseline architecture because it keeps the study focused on the data, preserves fine boundary detail through skip connections, works for both the smallest and the largest training set, is a recognised reference in building extraction, and is practical to train on one GPU. Its pre-trained ResNet-34 variant is kept as a controlled comparison rather than as the baseline. A complete, parameterised pipeline covering data loading, a BCE+Dice training objective, validation-based model selection, cross-dataset testing and result aggregation has been set up, together with explicit rules that keep the cross-dataset comparison fair.

Next steps: Milestone 4 trains the initial models on this pipeline, tunes hyper-parameters, optimisation methods and regularisation, and reports the measured results.

Contribution Log

Maintained for every milestone, as required in the Milestone 1 review. Each member confirms their own entry.

Team member	Contribution in this milestone(Section)
Bhawani Dutt	1,2,3
Jaival	4,5
Prakhar Mishra	4,5
Atharva Sarbhukan	6
Abhishek Rawat	7,8

Sign-off

Each team member confirms that they have reviewed and approved this document.

Team member	Reviewed and approved	Date
Bhawani Dutt	Approved	05-11-2026
Jaival	Approved	05-11-2026
Prakhar Mishra	Approved	05-11-2026
Atharva Sarbhukan	Approved	05-11-2026
Abhishek Rawat	Approved	05-11-2026

A How to Run the Code

```
# 1. install PyTorch for your CUDA version, then
pip install -r requirements.txt

# 2. sanity checks (pipeline acceptance, Section 6)
python check_data.py --data_root /path/prepared_dataset
python train.py --data_root /path/prepared_dataset --train_set whu --out runs/dry --dry_run

# 3. evaluate any checkpoint on any split
python evaluate.py --checkpoint runs/<run>/best.pt --data_root /path/prepared_dataset --split test
```

If the GPU runs out of memory, use `--batch_size 4` or `--crop 256`. Training commands for the experiments are given in the Milestone 4 report.

References

[1] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. *MICCAI 2015*, LNCS 9351, 234–241.

[2] Long, J., Shelhamer, E., & Darrell, T. (2015). Fully convolutional networks for semantic segmentation. *CVPR*, 3431–3440.

[3] Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., & Adam, H. (2018). Encoder-decoder with atrous separable convolution for semantic image segmentation. *ECCV*, 801–818.

[4] Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J. M., & Luo, P. (2021). SegFormer: Simple and efficient design for semantic segmentation with transformers. *NeurIPS*, 34.

[5] Dosovitskiy, A., et al. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*.

[6] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR*, 770–778.

[7] Iglovikov, V., & Shvets, A. (2018). TernausNet: U-Net with VGG11 encoder pre-trained on ImageNet for image segmentation. *arXiv:1801.05746*.

[8] Iakubovskii, P. (2019). Segmentation Models PyTorch. GitHub repository, https://github.com/qubvel/segmentation_models.pytorch.

[9] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *ICML*, 448–456.

[10] Milletari, F., Navab, N., & Ahmadi, S.-A. (2016). V-Net: Fully convolutional neural networks for volumetric medical image segmentation. *3DV*, 565–571.

[11] Ji, S., Wei, S., & Lu, M. (2019). Fully convolutional networks for multisource building extraction from an open aerial and satellite imagery data set. *IEEE TGRS*, 57(1), 574–586.

[12] Maggiori, E., Tarabalka, Y., Charpiat, G., & Alliez, P. (2017). Can semantic labeling methods generalize to any city? The Inria aerial image labeling benchmark. *IEEE IGARSS*, 3226–3229.

[13] Mnih, V. (2013). *Machine Learning for Aerial Image Labeling*. PhD thesis, University of Toronto.